

Methodology

Search space

The evaluated TinyLlama experiment used two pipeline stages with four GPUs per stage. Each stage selected one of three intra-stage strategies:

- TP1_DP4
- TP2_DP2
- TP4_DP1

The two stage choices produce nine strategy pairs. A 22-layer model has 21 non-empty two-stage partitions, so the recorded search space contained $9 \times 21 = 189$ configurations.

Performance estimation

The research workflow separated stage work from pipeline receive/wait time and constructed a profiling-derived cost for each candidate. The public [MarkovianBlockEstimator](#) captures the intended repeated-block abstraction using measurements of one block and two adjacent blocks:

```
marginal_block_cost = two_block_time - one_block_time
predicted_stage(n)  = one_block_time + (n - 1) * marginal_block_cost
                    + boundary_cost + communication_cost
```

This is a transparent approximation for a model built from repeated Transformer blocks. It assumes the marginal cost remains stable as blocks are added. Embedding, output-head, communication, and other boundary costs must be modeled separately.

The archived 5.85% result came from the original GPU prototype's profiling-derived cost pipeline. Raw per-rank logs and the upstream training runtime are intentionally excluded from this public-safe reconstruction, so this repository does not claim to reproduce that GPU measurement from scratch.

Dynamic-programming objective

For stage costs $c_1 \dots c_s$ and B microbatches, the search objective is:

```
latency = sum(c_i) + (B - 1) * max(c_i)
```

The sum captures pipeline fill and drain. The maximum captures the steady-state bottleneck. For each distinct candidate stage cost, the search treats it as a bottleneck threshold and performs an additive dynamic program over layer boundaries and device count.

Error definition

The reported aggregate uses measured net time as the denominator:

```
absolute percentage error = abs(actual_net - predicted) / actual_net * 100
```

This definition is stated explicitly because an earlier experiment CSV also retained a differently normalized **Diff_Percent** column. The public summary is recomputed from the raw measured and predicted columns rather than trusting that derived column.